Week-1

Writing Secure Code — Introduction

---

1. Introduction to Secure Coding

Software today is more connected, more complex, and more exposed to attackers than at any point in history. Applications interact with databases, cloud APIs, user inputs, networks, browsers, mobile devices and multiple unknown sources. Every point of interaction becomes a potential point of attack.

Secure coding is the practice of writing software that continues to work correctly even when used in unexpected, hostile, or malicious environments.

It's not just about making software "work"—
👉 it is about making software **safe**, **reliable**, and **resilient**.

Secure coding is needed in *every* environment:

- banking systems
- healthcare systems
- government portals
- e-commerce platforms
- mobile apps
- embedded systems
- cloud microservices
- IoT devices

A single insecure line of code can compromise an entire organization.

---

2. Why Do We Study Writing Secure Code?

Reason 1: Most security attacks come from programming mistakes

Studies show **70–80% of cyber attacks** happen due to software bugs caused by bad coding practices.

Example:
1990s–2000s Windows worm outbreaks (Code Red, Nimda, Blaster) exploited simple coding mistakes like buffer overflows.

Reason 2: Real organizations lose millions due to insecure code

- Equifax breach (2017) — caused by unpatched insecure code → 147 million records leaked.
- British Airways (2018) — insecure JavaScript code on payment portal → £183M GDPR fine.
- Hospitals attacked by ransomware → Weak input validation in remote desktops.

Reason 3: To produce job-ready software engineers

Companies expect graduates to:

- write input-validated code
- avoid SQL injection
- manage sessions safely
- use secure libraries
- understand OWASP Top 10 vulnerabilities

Reason 4: Prevention is cheaper than fixing

Fixing a bug in production is **100x costlier** than preventing it during development.

---

3. What Is a Vulnerability?

A **vulnerability** is a weakness in code that an attacker can exploit.

Common causes:

- trusting user input
- unsafe memory usage (C/C++)
- poor API usage
- weak encryption or no encryption
- poor authentication logic
- misconfigured servers
- weak error handling
- insecure defaults

A secure developer must always assume:
"All input is malicious until proven safe."

---

4. Principles of Secure Coding

4.1 Principle of Least Privilege

A program should only have the permissions absolutely necessary.

Example

A photo viewer app should not run with Administrator rights.
A database user for login page should only have SELECT privilege.

4.2 Validate Input Strictly

All user input must be:

- checked
- sanitized
- validated

Use whitelisting instead of blacklisting.

Example:

```
if(!username.matches("[A-Za-z0-9_]{3,20}"))
   throw new Error("Invalid username");
```

4.3 Use Secure Libraries

Avoid writing your own:

- encryption
- session management
- authentication

Always use vetted, community-tested libraries.

4.4 Fail Securely

If something fails, it must fail **safe**, not **open**.

Example

If authentication module fails, user should be **denied**, not **allowed**.

4.5 Defense in Depth

Multiple layers of security:

- Input validation
- Authentication
- Authorization
- Logging
- HTTPS
- Firewalls
- Secure coding

Secure code is not optional — it's essential.

Key Points:

- Most attacks exploit poor coding practices
- Secure coding prevents financial, reputational, and operational losses
- Always validate input
- Use safe APIs
- Understand common vulnerability categories
- Follow secure SDLC
- Think like an attacker
- Implement defense-in-depth

Week-2

2. Introduction to various Tools and Libraries to write secure code.

Writing secure code is an essential part of modern software development. The goal is to ensure that the software you build is free from security vulnerabilities, is maintainable, and adheres to best practices. Various tools and libraries are available to help developers write secure code by identifying vulnerabilities, enforcing security best practices, and assisting with secure code management.

These tools can be categorized into the following groups:

1. **Static Analysis Tools:** Analyze the source code without executing it to identify potential vulnerabilities early in the development cycle.
2. **Dependency Scanning Tools:** Help identify vulnerabilities in third-party libraries and components that your project depends on.
3. **Secure Coding Libraries:** Provide pre-built, secure implementations of cryptographic, authentication, and input validation routines.

A. Static Analysis Tools : Static analysis tools analyze the source code to detect potential security vulnerabilities, coding errors, and performance issues without executing the code. These tools can be integrated into your development process to continuously check your codebase for security flaws.

1. SonarQube : SonarQube is an open-source static analysis tool that helps detect bugs, security vulnerabilities, and code quality issues.

- Key Features:
  - Detects security flaws such as SQL injection, XSS, and buffer overflows.
  - Supports many programming languages (Java, C/C++, JavaScript, Python, etc.).
  - Provides detailed analysis and integrates with CI/CD pipelines.
- Advantages:

- ◦ Comprehensive analysis for a wide variety of languages.
- ◦ Provides actionable feedback and suggestions for code improvements.

• Disadvantages:

- ◦ Can be resource-intensive, especially for large projects.
- ◦ Setup can be complex for beginners, especially with large codebases.

2. Synopsys Coverity : Coverity is a powerful static code analysis tool used to find defects and security vulnerabilities in the code before it's compiled or executed.

• Key Features:

- ◦ Detects a range of vulnerabilities including buffer overflows, memory leaks, and null pointer dereferencing.
- ◦ Provides a detailed and prioritized list of issues based on risk level.
- ◦ Supports multiple languages like C/C++, Java, and more.

• Advantages:

- ◦ Effective for large and complex codebases, including third-party dependencies.
- ◦ Excellent for maintaining code quality and security in large teams.

• Disadvantages:

- ◦ Expensive, especially for small teams or startups.
- ◦ Requires expertise to configure and optimize.

3. DeepSource : DeepSource is an automated code review tool that helps identify security flaws, performance issues, and maintainability problems.

• Key Features:

- ◦ Detects common vulnerabilities like SQL injections and XSS.
- ◦ Supports multiple languages including Python, JavaScript, Go, Ruby, and more.
- ◦ Automatically reviews pull requests and provides suggestions.

• Advantages:

- ◦ Easy to set up and integrates seamlessly with version control systems like

GitHub, GitLab, and Bitbucket.

◦ Offers a user-friendly interface with detailed reports.

• Disadvantages:

◦ Limited language support (mainly focused on Python, Go, Ruby, JavaScript, etc.).

◦ May not detect more complex vulnerabilities that require deeper analysis.

4. Visual Code Grep : Visual Code Grep is a search extension for Visual Studio Code that allows you to search the codebase for specific patterns that might indicate security vulnerabilities or issues.

• Key Features:

◦ Enables deep search for patterns like hardcoded credentials, insecure HTTP headers, etc.

◦ Simple to use directly within the Visual Studio Code environment.

• Advantages:

◦ Lightweight and easy to use, particularly for smaller projects.

◦ Helps identify certain types of vulnerabilities like improper validation or insecure configurations.

◦ Integrates seamlessly with the Visual Studio Code IDE.

• Disadvantages:

◦ Limited compared to full-fledged static analysis tools.

◦ Requires manual review of results, which can be time-consuming for larger codebases.

◦ Not suited for identifying complex or hidden vulnerabilities.

B. Dependency Scanning Tools : Modern applications often rely on third-party libraries and components. If these libraries contain vulnerabilities, they can compromise the security of the entire application. Dependency scanning tools help identify such issues.

1. OWASP Dependency-Check : Dependency-Check is an open-source tool that identifies known vulnerabilities in your project's dependencies by cross-referencing them against public vulnerability databases.

- Key Features:
  - Scans both direct and transitive dependencies.
  - Cross-references libraries against public databases like the National Vulnerability Database (NVD).
  - Generates detailed reports that highlight vulnerable dependencies.
- Advantages:
  - Free and open-source, with a wide range of supported languages and frameworks.
  - Helps ensure you're not using libraries with known security vulnerabilities.
  - Integrates with common build systems like Maven and Gradle.
- Disadvantages:
  - Dependency resolution can be slow, especially in large projects.
  - Requires manual review of vulnerability reports to assess the severity of issues.

2. Snyk : Snyk is a vulnerability scanning tool for open-source dependencies, container images, and Infrastructure-as-Code (IaC) files.

- Key Features:
  - Detects vulnerabilities in open-source libraries and suggests fixes.
  - Provides real-time monitoring of dependencies and sends alerts when new vulnerabilities are discovered.
  - Offers automatic pull requests for vulnerable dependencies.
- Advantages:
  - Provides real-time vulnerability alerts and automatic fixes.
  - Easy integration with GitHub, GitLab, and other version control systems.
  - Supports both code dependencies and container security.
- Disadvantages:

◦ Can be expensive for large teams or projects with many dependencies.

◦ Limited language support compared to other tools like Dependency-Check.

C. Secure Coding Libraries : Secure coding libraries provide pre-built, security-focused code that developers can use to avoid common pitfalls like improper encryption, insecure input validation, and faulty authentication mechanisms.

1. Bouncy Castle : Bouncy Castle is a cryptographic library for Java and C# that provides a wide array of cryptographic algorithms.

• Key Features:

◦ Supports encryption, hashing, signing, and certificate management.

◦ Provides both high-level and low-level cryptographic operations.

• Advantages:

◦ Comprehensive and widely-used cryptographic library.

◦ Open-source and actively maintained.

◦ Well-documented and supported by a large community.

• Disadvantages:

◦ The API can be complex, especially for beginners.

◦ The library is extensive, and developers may need to focus on just the relevant parts for their use case.

3. Secure Software Tools Installation.

If you are using windows and want to set up ‘gcc’ to compile your code, you need to install ‘MinGW (Minimalist GNU for windows).

• You can download it from https://sourceforge.net/projects/mingw/

• Once downloaded, run the installer (mingw-get-setup.exe)

• After installation, open the MinGW installation manager

• In the package list, ensure the following are selected

• mingw32-base (for basic GCC)

• mingw32-gcc-g++ (for c++ support)

• Msys-base (for basic Unix-like utilities)

• Right click on each package, and select “mark for installation”.

• Then click Apply changes. (Appears when u exit)

• After applying changes, MinGW will download and install the required packages

• Add the bin folder (ex - c:\MinGW\bin) to the path (in system variables)

• You can now use gcc in your command prompt.

Week 5:

Implement a program to Avoiding Server Hijacking.

- A small TCP server that executes commands sent by a client
- Vulnerable version executes commands directly → attacker can hijack system
- Secure version validates commands → prevents hijacking

Vulnerable Server Code (allows server hijacking risk)

```
import java.io.*;

import java.net.*;

public class VulnerableServer {

   public static void main(String[] args) throws Exception {

      ServerSocket server = new ServerSocket(6003);

      System.out.println("Server started on port 6003");

      Socket socket = server.accept();

      BufferedReader in = new BufferedReader(

            new InputStreamReader(socket.getInputStream()));

      System.out.println("Enter the command: ");

      String command = in.readLine(); // receives command from user

      System.out.println("Received command: " + command);

      //  VERY VULNERABLE: executes user input directly

      Runtime.getRuntime().exec(command);

      System.out.println("Command executed (dangerous!)");

      socket.close();

      server.close();

   }

}
```

OUTPUT:

```
D:\AU\2025-26(II Sem)\WSC Lab>javac VulnerableServer.java
Note: VulnerableServer.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.

D:\AU\2025-26(II Sem)\WSC Lab>java VulnerableServer
Server started on port 6003
Enter the command:
Received command: notepad
Command executed (dangerous!)
```

Secure Server Code (prevents server hijacking)

We protect the server by:

- Allow only a **safe list of commands.**
- Reject unknown/malicious inputs.
- Never pass user input directly to exec.

```
import java.io.*;

import java.net.*;

import java.util.*;

public class SecureServer {

   private static final List<String> allowedCommands =

         Arrays.asList("TIME", "DATE", "HELLO");

   public static void main(String[] args) throws Exception {

      ServerSocket server = new ServerSocket(5000);

      System.out.println("Secure server on port 5000");

      Socket socket = server.accept();

      BufferedReader in = new BufferedReader(

            new InputStreamReader(socket.getInputStream()));

      String cmd = in.readLine();

      System.out.println("Received: " + cmd);

      if (!allowedCommands.contains(cmd)) {

         System.out.println("ERROR: Command rejected!");

      } else {

         if (cmd.equals("HELLO")) System.out.println("Hello User!");
```

```
        if (cmd.equals("TIME")) System.out.println(new Date().getTime());
        if (cmd.equals("DATE")) System.out.println(new Date().toString());
      }
      socket.close();
      server.close();
   }
}
```

OUTPUT:

```
D:\AU\2025-26(II Sem)\WSC Lab>javac SecureServer.java

D:\AU\2025-26(II Sem)\WSC Lab>java SecureServer
Secure server on port 5000
Received: notepad
ERROR: Command rejected!
```

```
D:\AU\2025-26(II Sem)\WSC Lab>javac SecureServer.java

D:\AU\2025-26(II Sem)\WSC Lab>java SecureServer
Secure server on port 5000
Received: HELLO
Hello User!
```

Write a program to Limiting the Domain Usage.

Vulnerable Code (no domain restrictions)

Note :- Program cannot block malicious domain.

```
import java.util.Scanner;
public class VulnerableDomain {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.println("Enter URL to connect:");
      String url = sc.nextLine();
      //  Vulnerable: accepts ANY domain
```

```
        System.out.println("Connecting to: " + url);
        // attacker can enter:
        // http://evil.com/malware
        // server will connect blindly
        sc.close();
    }
}
```

OUTPUT:

```
D:\AU\2025-26(II Sem)\WSC Lab>javac VulnerableDomain.java

D:\AU\2025-26(II Sem)\WSC Lab>java VulnerableDomain
Enter URL to connect:
http://evil.com
Connecting to: http://evil.com
```

Secure Code

We restrict domain access to safe trusted domains only.

```
import java.util.*;
public class SecureDomain {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // allowed safe domains only
        List<String> allowed = Arrays.asList(
                "example.com", "university.edu", "localhost");
        System.out.println("Enter domain to connect:");
        String domain = sc.nextLine();
        if (allowed.contains(domain)) {
            System.out.println("Connection allowed to: " + domain);
```

```
            // safe processed logic here
        } else {
            System.out.println("ERROR: Domain not allowed!");
        }
        sc.close();
    }
}
```

OUTPUT:

```
D:\AU\2025-26(II Sem)\WSC Lab>javac SecureDomain.java

D:\AU\2025-26(II Sem)\WSC Lab>java SecureDomain
Enter domain to connect:
http://evil.com
ERROR: Domain not allowed!

D:\AU\2025-26(II Sem)\WSC Lab>java SecureDomain
Enter domain to connect:
example.com
Connection allowed to: example.com
```

Write a program to test User Input Vulnerabilities.

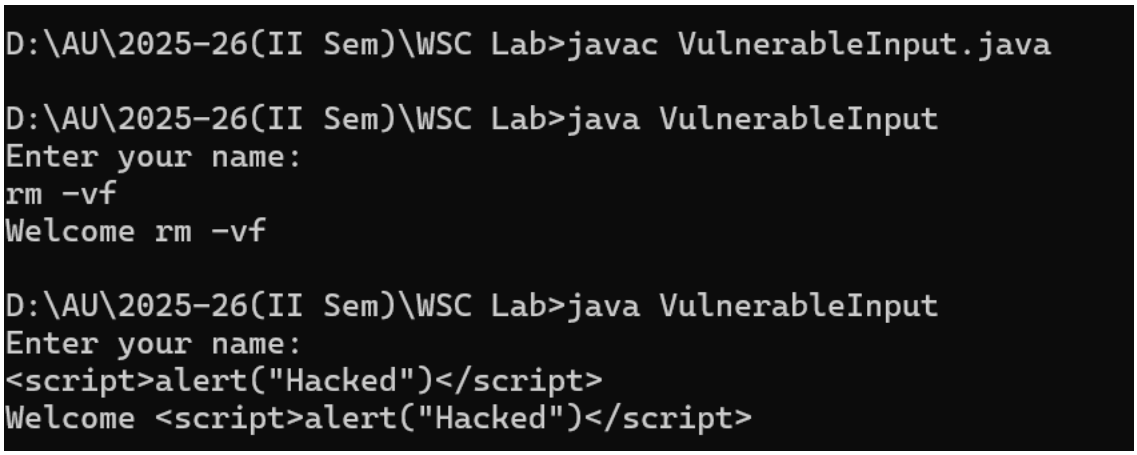
Vulnerable Program – No Input Validation

Note :- Program echoes the malicious payload as it is.

```
import java.util.Scanner;
public class VulnerableInput {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter your name:");
        String name = sc.nextLine();
        // Vulnerable: directly uses user input
```

```
        System.out.println("Welcome " + name);

        sc.close();

    }

}
```

OUTPUT:



Secure Program – Input Validation / Sanitization

```
import java.util.Scanner;
public class SecureInput {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter your name:");
        String name = sc.nextLine();
        // secure: remove dangerous characters <> and script keywords
        name = name.replaceAll("<", "")
                .replaceAll(">", "")
                .replaceAll("(?i)script", "");
        System.out.println("Welcome " + name);
        sc.close();
    }
}
```

OUTPUT:

```
D:\AU\2025-26(II Sem)\WSC Lab>javac SecureInput.java

D:\AU\2025-26(II Sem)\WSC Lab>java SecureInput
Enter your name:
Anurag University
Welcome Anurag University

D:\AU\2025-26(II Sem)\WSC Lab>java SecureInput
Enter your name:
<script>alert("Hacked")</script>
Welcome alert("Hacked")
```

Week 6

9. Implement SQL Injection technique.

SQL Injection is a vulnerability where **user input is concatenated directly into an SQL query**, allowing an attacker to **alter the query logic**.

Steps to execute:

- Install MySQL
- Create a database.
  CREATE DATABASE testdb;
- Change database
  USE testdb;
- Create a table in database
  ```
  CREATE TABLE users (
     id INT AUTO_INCREMENT PRIMARY KEY,
     username VARCHAR(50),
     password VARCHAR(50)
  );
  ```
- Insert atleast 5 rows in table.
  ```
  INSERT INTO users (username, password)
  VALUES ('admin', 'admin123');
  ```
- Download jar file. Follow the steps to download jar file

  - ✓ Open browser
  - ✓ Go to 👉 https://dev.mysql.com/downloads/connector/j/
  - ✓ Select **Operating System: Platform Independent**
  - ✓ Click **Download(ZIP file)**
  - ✓ Skip login → click **No thanks, just start my download**

- From the downloaded folder, place the executable jar file and the file in the same folder
- Then Compile:
  javac -cp .;mysql-connector-j-9.5.0.jar filename.java
- Then Run:
  java -cp .;mysql-connector-j-9.5.0.jar filename
- Enter Username and Password as per your table entries.

<u>Vulnerable code:</u>

```
import java.sql.*;

import java.util.Scanner;

public class SQLInjectionDemo {
```

```
    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        System.out.print("Username: ");
        String user = sc.nextLine();
        System.out.print("Password: ");
        String pass = sc.nextLine();
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/testdb",
            "root",
            "root"
        );
        Statement stmt = con.createStatement();
        // VULNERABLE QUERY
        String query =
            "SELECT * FROM users WHERE username='" + user +
            "' AND password='" + pass + "'";
        ResultSet rs = stmt.executeQuery(query);
        if (rs.next())
            System.out.println("Login Successful");
        else
            System.out.println("Login Failed");
        con.close();
        sc.close();
    }
}
```

Compile:

```
javac -cp .;mysql-connector-j-9.5.0.jar SQLInjectionDemo.java
```

Run:

java -cp .;mysql-connector-j-9.5.0.jar SQLInjectionDemo

OUTPUT:

```
D:\AU\2025-26(II Sem)\WSC Lab>javac -cp .;mysql-connector-j-9.5.0.jar SQLInjectionDemo.java

D:\AU\2025-26(II Sem)\WSC Lab>java  -cp .;mysql-connector-j-9.5.0.jar SQLInjectionDemo
Username: admin
Password: admin123
Login Successful

D:\AU\2025-26(II Sem)\WSC Lab>java  -cp .;mysql-connector-j-9.5.0.jar SQLInjectionDemo
Username: admin
Password: ' OR '1'='1
Login Successful
```

Safe Code:

```
import java.sql.*;
import java.util.Scanner;
public class SecureSQLInjection {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Username: ");
        String username = sc.nextLine();
        System.out.print("Password: ");
        String password = sc.nextLine();

        try {
            // Load MySQL JDBC Driver
            Class.forName("com.mysql.cj.jdbc.Driver");
            // Database connection
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/testdb",
                "root",
                "root"   // your MySQL password
            );
```

```
        // SECURE QUERY (NO SQL INJECTION)
        String query =
           "SELECT * FROM users WHERE username = ? AND password = ?";
        PreparedStatement ps = con.prepareStatement(query);
        ps.setString(1, username);
        ps.setString(2, password);
        ResultSet rs = ps.executeQuery();
        if (rs.next()) {
           System.out.println("Login Successful");
        } else {
           System.out.println("Login Failed");
        }
        con.close();
        sc.close();
      } catch (Exception e) {
        e.printStackTrace();
      }
   }
}
```

Compile:

javac -cp .;mysql-connector-j-9.5.0.jar SecureSQLInjection.java

Run:

java -cp .;mysql-connector-j-9.5.0.jar SecureSQLInjection

OUTPUT:

```
D:\AU\2025-26(II Sem)\WSC Lab>javac -cp .;mysql-connector-j-9.5.0.jar SecureSQLInjection.java

D:\AU\2025-26(II Sem)\WSC Lab>java  -cp .;mysql-connector-j-9.5.0.jar SecureSQLInjection
Username: admin
Password: admin123
Login Successful

D:\AU\2025-26(II Sem)\WSC Lab>java  -cp .;mysql-connector-j-9.5.0.jar SecureSQLInjection
Username: admin
Password: ' OR '1'='1
Login Failed
```

frame?

A frame (iframe) is a box inside a webpage that loads another webpage.

X-Frame-Options:

X-Frame-Options is an HTTP response security header used to prevent clickjacking attacks by controlling whether a web page can be displayed inside an <iframe>.

Clickjacking:

A clickjacking attack tricks users into clicking hidden or transparent frames that load another website.

Vulnerable Code(No X-Frame-Options):

This program creates a simple web server using Java that serves a webpage without X-Frame-Options, making it vulnerable to clickjacking. No protection against clickjacking. Page can be embedded inside an iframe

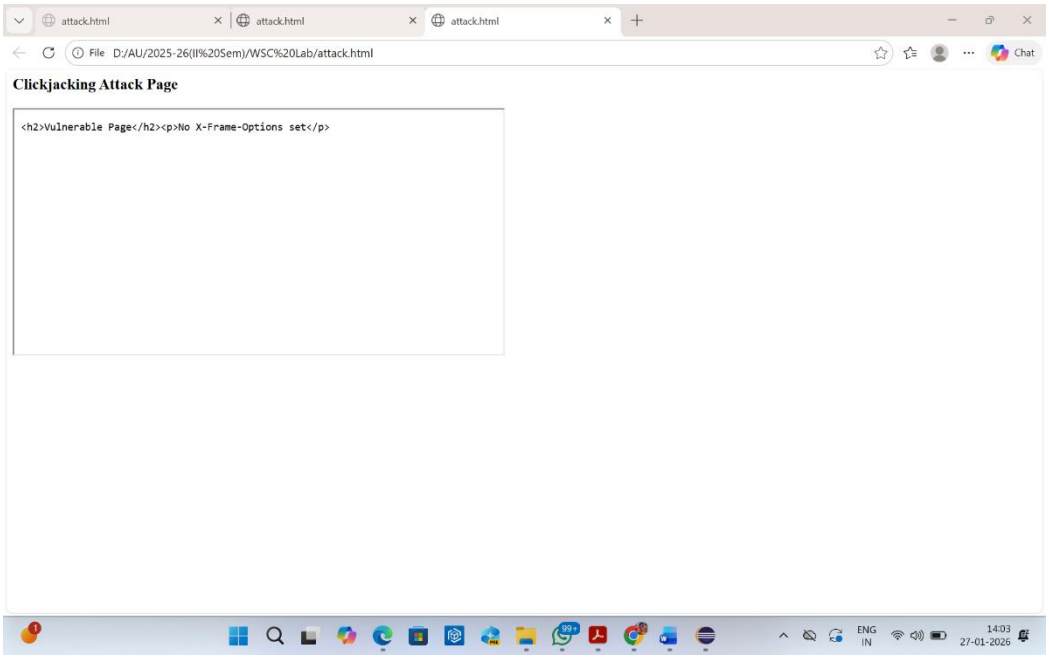
```
import com.sun.net.httpserver.*;
import java.io.*;
import java.net.InetSocketAddress;
public class VulnerableXFrame {
   public static void main(String[] args) throws Exception {
       HttpServer server = HttpServer.create(
             new InetSocketAddress(8000), 0);
       server.createContext("/", exchange -> {

          String response = "<h2>Vulnerable Page</h2>"
                + "<p>No X-Frame-Options set</p>";
          exchange.sendResponseHeaders(200,
response.length());
```

```
            OutputStream os = exchange.getResponseBody();
            os.write(response.getBytes());
            os.close();
        });
        server.start();
        System.out.println("Vulnerable server running at http://localhost:8000");
    }
}
```

Steps to execute:

- Compile: javac VulnerableXFrame.java
- Run: java VulnerableXFrame
- Vulnerable server running at http://localhost:8000
- Open http://localhost:8000
- Clickjacking Test:
- Create attack.html:

  ```
  <h3>Clickjacking Attack Page</h3>
  <iframe src="http://localhost:8000" width="600" height="300"></iframe>
  ```
- Page loads inside iframe



Secure Code (X-Frame-Options Enabled)

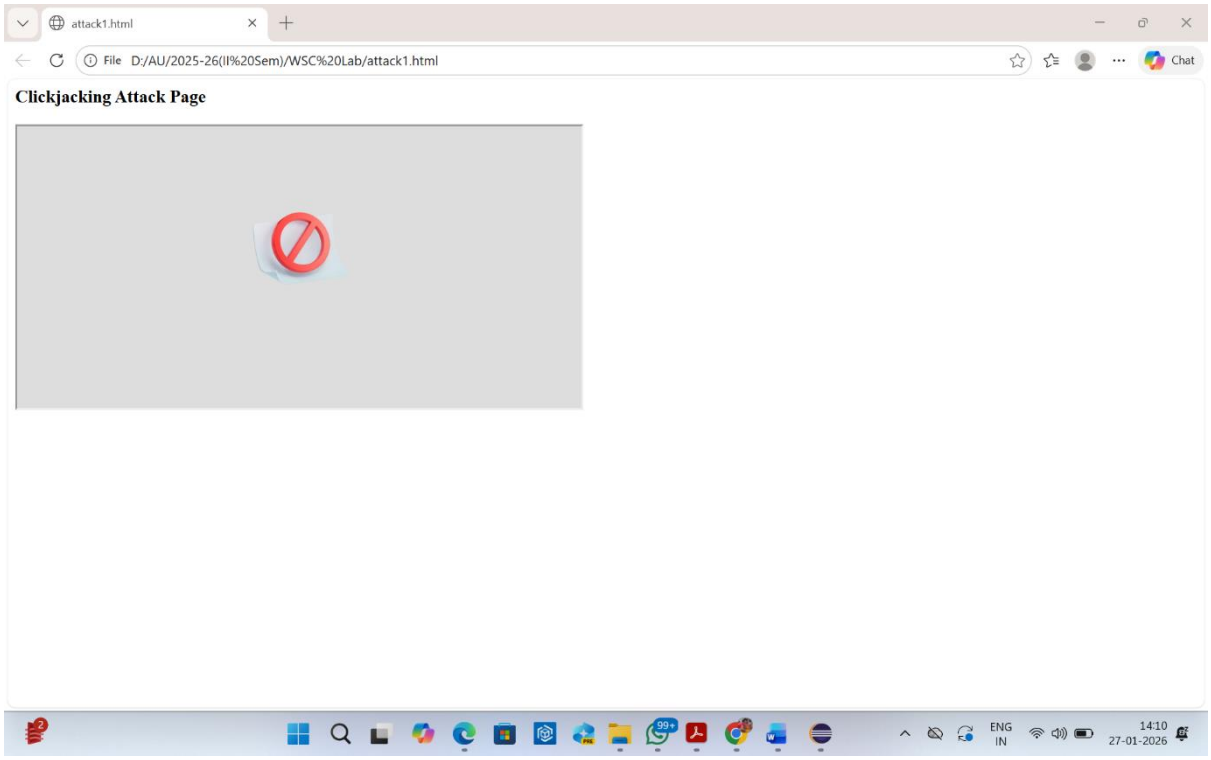
- Prevents iframe embedding
- Stops clickjacking attacks

```
import com.sun.net.httpserver.*;
import java.io.*;
import java.net.InetSocketAddress;
public class SecureXFrame {
   public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(
              new InetSocketAddress(9000), 0);
        server.createContext("/", exchange -> {
           // SECURITY HEADER
           exchange.getResponseHeaders()
                 .add("X-Frame-Options", "DENY");
```

```
            // or SAMEORIGIN
            String response = "<h2>Secure Page</h2>"
                    + "<p>X-Frame-Options Enabled</p>";
            exchange.sendResponseHeaders(200,
response.length());
            OutputStream os =
exchange.getResponseBody();
            os.write(response.getBytes());
            os.close();
        });
        server.start();
        System.out.println("Secure server running at
http://localhost:9000");
    }
}
```

Steps to execute:

- Compile: javac SecureXFrame.java
- Run: java SecureXFrame
  Secure server running at http://localhost:9000
- Open http://localhost:9000
- Clickjacking Test:
  - Create attack.html:
    <h3>Clickjacking Attack Page</h3>
    <iframe src="http://localhost:9000"
    width="600" height="300"></iframe>
- Page does not load inside iframe


attack1.html
File D:/AU/2025-26(II%20Sem)/WSC%20Lab/attack1.html
Chat
Clickjacking Attack Page
ENG
IN
14:10
27-01-2026

11. Write a program to implement HTTP security headers.

HTTP Security Headers

HTTP security headers are used by the **server** to tell the **browser** how to behave securely. They help prevent attacks like:

- Cross-Site Scripting (XSS)
- Clickjacking
- MIME-type attacks
- Data leakage
- Man-in-the-middle attacks

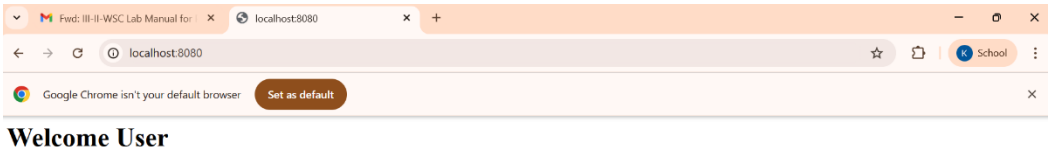
Vulnerable Java Program (No Security Headers)

This program creates a simple HTTP server but does **not use any security headers**.

```
import com.sun.net.httpserver.*;
import java.io.*;
import java.net.InetSocketAddress;
public class VulnerableHTTPServer {
   public static void main(String[] args) throws Exception {
      HttpServer server = HttpServer.create(new InetSocketAddress(8080), 0);
      server.createContext("/", exchange -> {
         String response = "<h1>Welcome User</h1>";
         exchange.sendResponseHeaders(200, response.length());
         OutputStream os = exchange.getResponseBody();
         os.write(response.getBytes());
         os.close();
      });
      server.start();
      System.out.println("Vulnerable server running on http://localhost:8080");
   }
}
```

Steps to Execute:

- Compile: javac VulnerableHTTPServer.java
- Run: java VulnerableHTTPServer
  Vulnerable server running on http://localhost:8080
- Open http://localhost:8080



Secure Java Program (With HTTP Security Headers)

Now the same program with **proper security headers added**.

```
import com.sun.net.httpserver.*;
import java.io.*;
import java.net.InetSocketAddress;
public class SecureHTTPServer {
    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(new InetSocketAddress(8080), 0);
        server.createContext("/", exchange -> {
            // ADD SECURITY HEADERS
            exchange.getResponseHeaders().add("X-Content-Type-Options", "nosniff");
            exchange.getResponseHeaders().add("X-Frame-Options", "DENY");
            exchange.getResponseHeaders().add("X-XSS-Protection", "1; mode=block");
            exchange.getResponseHeaders().add("Content-Security-Policy", "default-src 'self'");
            exchange.getResponseHeaders().add("Strict-Transport-Security", "max-
age=31536000; includeSubDomains");
            String response = "<h1>Secure Welcome</h1>";
            exchange.sendResponseHeaders(200, response.length());
            OutputStream os = exchange.getResponseBody();
            os.write(response.getBytes());
            os.close();
        });
        server.start();
        System.out.println("Secure server running on http://localhost:8080");
    }
```

}

Steps to Execute:

- Compile: javac VulnerableHTTPServer.java
- Run: java VulnerableHTTPServer
  Vulnerable server running on http://localhost:8080
- Open http://localhost:8080

12. Write a program to implement HTTP Cookies.

HTTP Cookies

An **HTTP cookie** is a small piece of data stored in the **user's browser** by the **server**.

It helps the server:

- Remember the user
- Maintain login session
- Store preferences (like theme, language)
- Track visits

Vulnerable Code: Without Cookies (Server forgets the user)

Every time user visits, server treats them as **new**.

```
import com.sun.net.httpserver.*;
import java.io.*;
import java.net.InetSocketAddress;
public class NoCookieServer {
    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(new InetSocketAddress(8000), 0);
```

```
        server.createContext("/home", exchange -> {
            String response = "<h2>Welcome Guest</h2>";
            exchange.sendResponseHeaders(200, response.length());
            OutputStream os = exchange.getResponseBody();
            os.write(response.getBytes());
            os.close();
        });
        server.start();
        System.out.println("Server running at http://localhost:8000/home");
    }
}
```

Steps to execute :

- Compile: javac NoCookieServer.java
- Run: java NoCookieServer
- Input (in browser): http://localhost:8000/home

Output:

Welcome Guest

Note: The browser displays **"Welcome Guest"** every time, indicating that the server does not retain user information.

Secure Code: With Cookies (Server remembers the user)

Now server stores a cookie: `username=Student`

```
import com.sun.net.httpserver.*;
import java.io.*;
import java.net.InetSocketAddress;
import java.util.List;
public class CookieServer {
    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(new InetSocketAddress(8000), 0);
        server.createContext("/home", exchange -> {
```

```
        String username = null;
        // Step 1: Read cookies sent by browser
        List<String> cookies = exchange.getRequestHeaders().get("Cookie");
        if (cookies != null) {
            for (String cookie : cookies) {
                if (cookie.contains("username=")) {
                    username = cookie.split("username=")[1];
                }
            }
        }
        String response;
        // Step 2: If no cookie found, set new cookie
        if (username == null) {
            exchange.getResponseHeaders().add("Set-Cookie", "username=Student");
            response = "<h2>Hello, new user! Cookie stored.</h2>";
        } else {
            response = "<h2>Welcome back, " + username + "</h2>";
        }
        exchange.sendResponseHeaders(200, response.length());
        OutputStream os = exchange.getResponseBody();
        os.write(response.getBytes());
        os.close();
      });
      server.start();
      System.out.println("Cookie server running at http://localhost:8000/home");
   }
}
```

First Visit

- The browser sends a request to the server for the `/home` page.
- The server sends the response page along with a cookie `Set-Cookie: username=Student`.
- The browser stores the cookie on the client side.

Second Visit

- The browser again sends a request for the /home page.
- The stored cookie (username=Student) is sent along with the request.
- The server reads the cookie and recognizes the user.
- The server sends a personalized response to the browser.

OUTPUT:

- **Compile: javac CookieServer.java**

- **Run: java CookieServer**
- Open browser: **http://localhost:8000/home**
- **First time accessing the URL:**
  Hello, new user! Cookie stored.
- **On refreshing the page:**
  Welcome back, Student.
- Stop server: **Ctrl + C**

13. Write a program for Testing Sockets-Based Applications.

Socket programs allow **communication between two programs** over a network.

Examples:

- Chat applications
- Client–server systems
- Online games
- File transfer
- Banking systems

Testing is required to ensure:

- Invalid input does not crash the server
- Unauthorized clients are rejected
- Data is validated
- Connection is handled safely

Vulnerable Socket Program (Unsafe Server)

This server accepts **any input blindly**.

Server (VulnerableServer.java)

```
import java.net.*;
import java.io.*;
public class VulnerableServer {
   public static void main(String[] args) throws Exception {
      ServerSocket serverSocket = new ServerSocket(5000);
      System.out.println("Vulnerable Server started on port 5000");
      Socket socket = serverSocket.accept(); // Accept any client
      BufferedReader in = new BufferedReader(
            new InputStreamReader(socket.getInputStream()));
      String message = in.readLine();  // No validation
      System.out.println("Client says: " + message);

      socket.close();
      serverSocket.close();
   }
}
```

Client (Client.java)

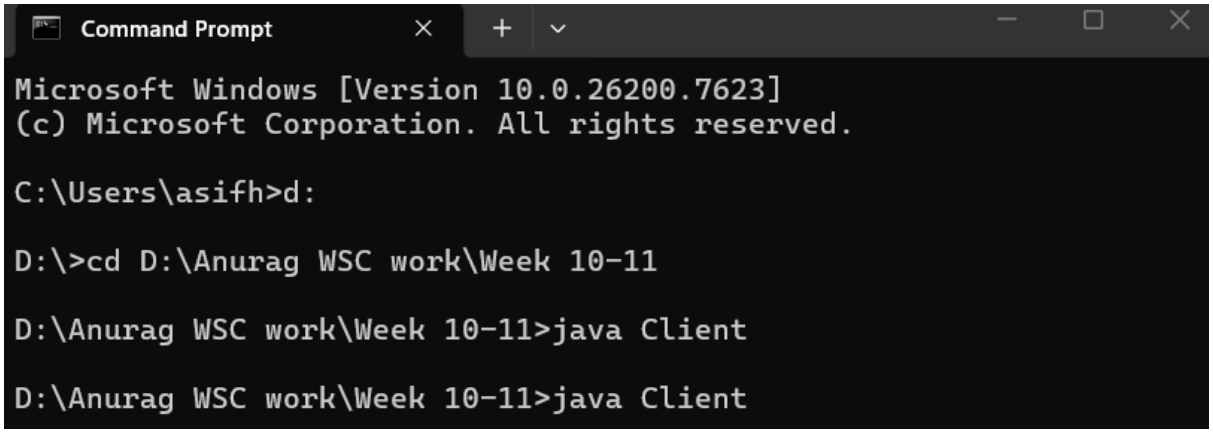
```
import java.net.*;
```

```
import java.io.*;
public class Client {
   public static void main(String[] args) throws Exception {
      Socket socket = new Socket("localhost", 5000);
      PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
      out.println("Hello Server");
      socket.close();
   }
}
```

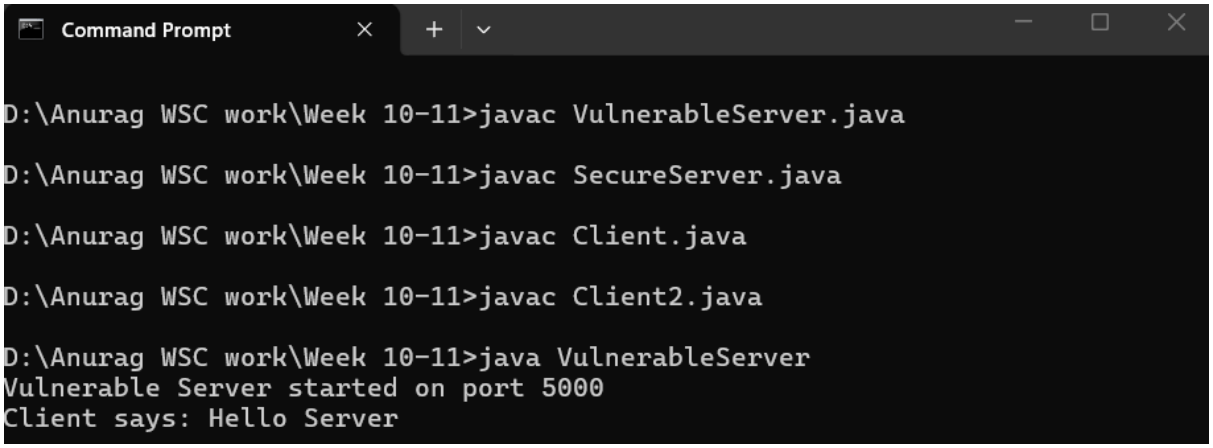
Problem in Vulnerable Version

- Any client can connect
- Malicious input can crash server
- No validation
- No restriction
- Unsafe for real applications

Client Side Output:



Server Side Output:



Secure Socket Program (Tested + Validated)

Now we **test and protect the socket program**:

- Reject empty messages
- Reject long messages
- Allow only valid characters

```
import java.net.*;
import java.io.*;
public class SecureServer {
   public static void main(String[] args) throws Exception {
      ServerSocket serverSocket = new ServerSocket(5000);
      System.out.println("Secure Server started on port 5000");
      Socket socket = serverSocket.accept();
      BufferedReader in = new BufferedReader(
            new InputStreamReader(socket.getInputStream()));
      PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
      String message = in.readLine();
      // Testing + Validation
      if (message == null || message.trim().isEmpty()) {
         out.println("Rejected: Empty message");
      }
      else if (message.length() > 50) {
         out.println("Rejected: Message too long");
      }
      else if (!message.matches("[a-zA-Z0-9 ]+")) {
         out.println("Rejected: Invalid characters");
      }
      else {
         out.println("Message accepted: " + message);
      }

      socket.close();
      serverSocket.close();
   }
}
```

Client (same as before)

```
import java.net.*;
import java.io.*;
public class Client {
   public static void main(String[] args) throws Exception {
```

```
        Socket socket = new Socket("localhost", 5000);
        PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
        BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));
        out.println("Hello Server");
        System.out.println("Server response: " + in.readLine());
        socket.close();
    }
}
```

Client side Output:

```
D:\Anurag WSC work\Week 10-11>java Client2
Server response: Message accepted: Hello Server

D:\Anurag WSC work\Week 10-11>java Client2
Server response: Rejected: Invalid characters

D:\Anurag WSC work\Week 10-11>
```

Server Side Output:

```
D:\Anurag WSC work\Week 10-11>java SecureServer
Secure Server started on port 5000

D:\Anurag WSC work\Week 10-11>java SecureServer
Secure Server started on port 5000

D:\Anurag WSC work\Week 10-11>javac Client2.java

D:\Anurag WSC work\Week 10-11>java SecureServer
Secure Server started on port 5000

D:\Anurag WSC work\Week 10-11>
```

Write a program for Testing HTTP-Based Applications.

Testing HTTP-Based Applications

HTTP-based applications are **web applications** where:

- Browser (client) sends request
- Server (Java program) sends response

Testing ensures:

- Invalid input is rejected
- Attacks like XSS are blocked
- Server does not accept dangerous data

- Application behaves safely

Vulnerable HTTP Program (No Input Testing)

This program accepts **any user input**, including malicious script.

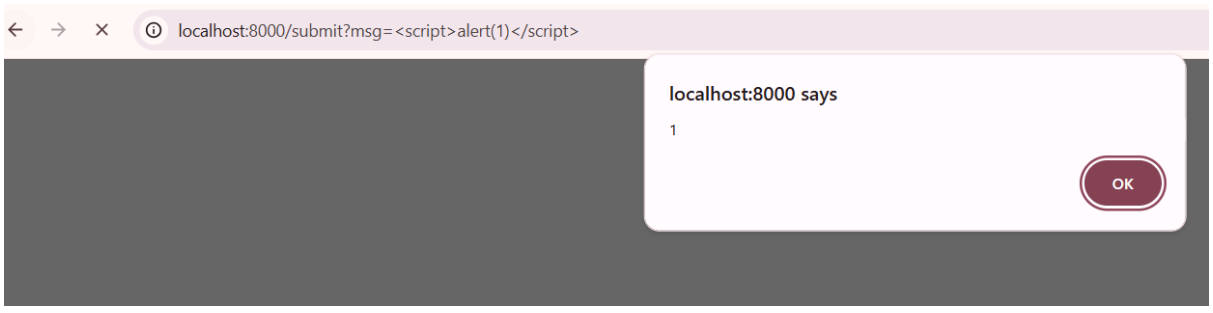
```
import com.sun.net.httpserver.*;
import java.io.*;
import java.net.InetSocketAddress;
public class VulnerableHttpServer {
    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(new InetSocketAddress(8000), 0);
        server.createContext("/submit", exchange -> {
            // Simulated user input (query string)
            String query = exchange.getRequestURI().getQuery();
            // Example: /submit?msg=Hello
            String response = "<h2>User Message: " + query + "</h2>";
            exchange.sendResponseHeaders(200, response.length());
            OutputStream os = exchange.getResponseBody();
            os.write(response.getBytes());
            os.close();
        });
        server.start();
        System.out.println("Vulnerable HTTP Server running at
http://localhost:8000/submit?msg=Hello");
    }
}
```

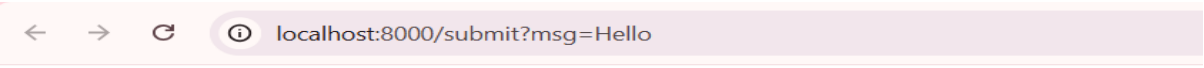
Problem here

If attacker enters:

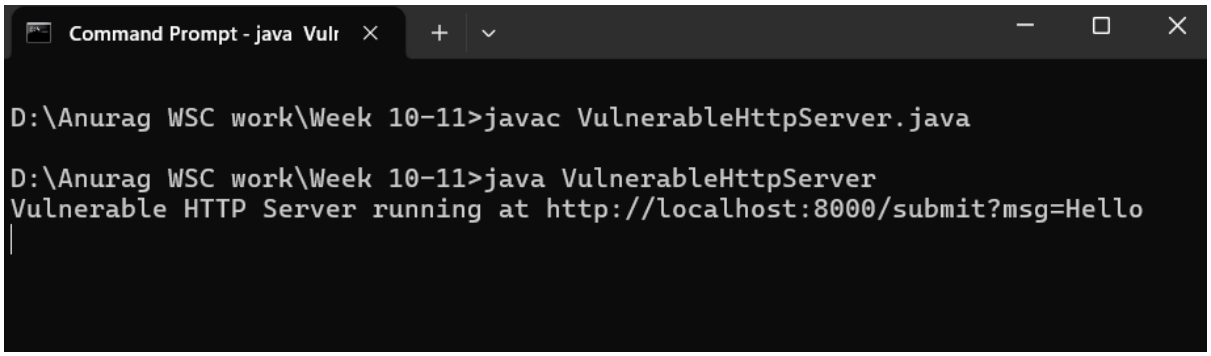
http://localhost:8000/submit?msg=<script>alert(1)</script>

The browser executes the script → XSS vulnerability





User Message: msg=Hello



Secure HTTP Program (With Testing & Validation)

Now we **test the input before accepting it**.

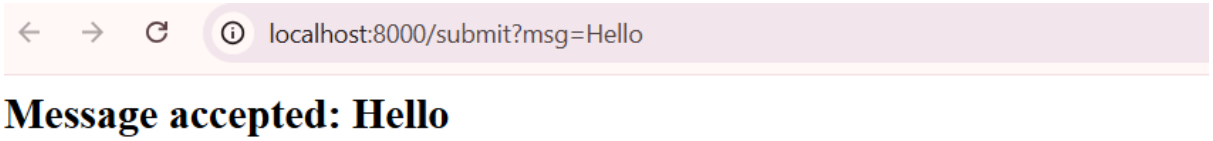
```
import com.sun.net.httpserver.*;
import java.io.*;
import java.net.InetSocketAddress;

public class SecureHttpServer {
    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(new InetSocketAddress(8000), 0);
        server.createContext("/submit", exchange -> {
            String query = exchange.getRequestURI().getQuery();
            String message = "";
            if (query != null && query.startsWith("msg=")) {
                message = query.substring(4);
            }
            String response;
            // Input Testing / Validation
            if (message.isEmpty()) {
                response = "<h2>Error: Empty input rejected</h2>";
            }
            else if (message.length() > 30) {
                response = "<h2>Error: Message too long</h2>";
```

```
        }
        else if (!message.matches("[a-zA-Z0-9 ]+")) {
            response = "<h2>Error: Invalid characters detected</h2>";
        }
        else {
            response = "<h2>Message accepted: " + message + "</h2>";
        }
        exchange.sendResponseHeaders(200, response.length());
        OutputStream os = exchange.getResponseBody();
        os.write(response.getBytes());
        os.close();
    });
    server.start();
    System.out.println("Secure HTTP Server running at
http://localhost:8000/submit?msg=Hello");
  }
}
```

OUTPUT:

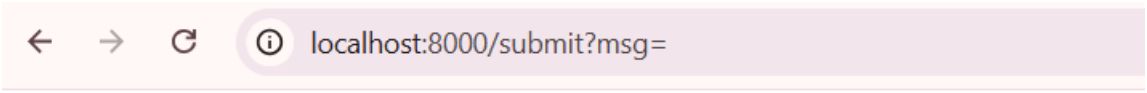
Input without Malicious Message:

- **Open browser:** http://localhost:8000/submit?msg=Hello Student



Input with Empty Message:

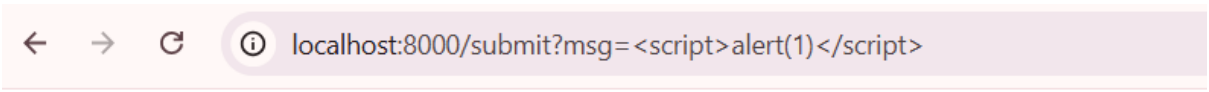
- **Open browser:** http://localhost:8000/submit?msg=



Error: Empty input rejected

Input with Malicious Message:

- **Open browser:** http://localhost:8000/submit?msg=<script>alert(1)</script>



Error: Invalid characters detected

```
import java.io.*;
public class FileBasedAppTest {
   public static void main(String[] args) {
            //File 1
            String f_Name = "data.txt";
            String f_Data = "Welcome to File Based Testing";
            //File 2
            String test_Name= "testdata.txt";
            String test_Data = "Welcome to File Based Testing";
try {
            FileWriter fw = new FileWriter(f_Name);
            fw.write(f_Data);
            fw.close();
            FileWriter fw1 = new FileWriter(test_Name);
            fw1.write(test_Data);
            fw1.close();
            BufferedReader br = new BufferedReader(new FileReader(f_Name));
            String readData = br.readLine();
            br.close();
            if (test_Data.equals(readData)) {
             System.out.println("TEST PASSED: Data matched.");
            } else {
             System.out.println("TEST FAILED: Data mismatch.");
             }
      } catch (IOException e) {
         System.out.println("TEST FAILED: File error occurred.");
      }
   }
}
```